Spring Core — IoC, Dependency Injection & Loosely Coupled Design

Topics Covered in This PDF

1. Why Spring Core is needed
2. Understanding Dependency
3. Tight Coupling
4. Tight Coupling in Code
5. What exactly is Tight Coupling?
6. Using an Interface to improve coupling
7. Why an Interface alone is not enough
8. Dependency Injection
9. Object creation outside the dependent class
10. What is Dependency Injection?
11. Benefits of Dependency Injection
12. Types of Dependency Injection
    - Constructor Injection
    - Setter Injection
    - Field Injection
13. What is IoC?
14. Simple definition of IoC
15. Relationship between IoC and DI
16. Where Spring fits in
17. Spring IoC Container
18. What Spring Core does
19. What is a Bean?
20. Manual Java Flow vs Spring Flow
21. Final summary and interview recall

1. Why Do We Need Spring Core?

Java already provides many powerful features:

- Classes
- Objects
- Constructors
- Interfaces
- Inheritance
- Packages

So the natural question is:

> **If Java already provides all these features, why do we need Spring Core?**

Spring Core is not mainly about building web applications. Web development comes later with technologies such as Spring MVC and Spring Boot.

The core idea of Spring is:

> **Spring helps us create objects, manage dependencies, and connect objects together in a clean and scalable way.**

In a small application, object creation is easy.

But in a real application:

```
Class A
   |
   v
depends on B
   |
   v
B depends on C
   |
   v
C depends on D
   |
   v
...
```

As the dependency chain grows, manually creating and connecting objects becomes difficult.

This is where **Spring Core** becomes useful.

---

2. Understanding Dependency

Suppose we have:

```
class EmailService {

    public void sendEmail() {
        System.out.println("Email sent");
    }
}
```

Now suppose `OrderService` needs to send an email after placing an order:

```
class OrderService {

    private EmailService emailService = new EmailService();

    public void placeOrder() {
        System.out.println("Order placed");
        emailService.sendEmail();
    }
}
```

Here:

```
OrderService
     |
     | needs
     v
EmailService
```

What is a dependency?

> **A dependency is something a class needs in order to complete its work.**

In this example:

```
private EmailService emailService = new EmailService();
```

means:

- `OrderService` needs `EmailService`.
- `OrderService` is creating its own dependency.

The dependency itself is normal.

The design problem is **how the dependency is created and controlled**.

---

3. The Problem — Tight Coupling

There are two broad design styles:

```
Tightly Coupled Design
        |
        v
Hard to change

Loosely Coupled Design
        |
        v
Easier to change
```

Real-Life Example

Suppose a person wants to travel from Delhi to Chandigarh.

Tightly coupled thinking

> "I can travel only by this one specific car, with this one specific driver, from this one specific company."

If that car is unavailable, the person is stuck.

Loosely coupled thinking

> "I need transportation."

Transportation could be:

```
Car
Bus
Train
Cab
```

The person cares about the **service/capability**, not one specific implementation.

Code analogy

```
Tight coupling:

OrderService
     |
     | specifically depends on
     v
EmailService


Loose coupling:

OrderService
     |
     | depends on abstraction
     v
NotificationService
     ^
     |
 +---+---------+
 |             |
EmailService  SmsService
```

---

4. Tight Coupling in Code

Original code:

```
class OrderService {

    private EmailService emailService = new EmailService();

    public void placeOrder() {
        System.out.println("Order placed");
        emailService.sendEmail();
    }
}
```

The relationship is:

```
OrderService
      |
      | directly creates
      v
EmailService
```

Now imagine the requirement changes.

Instead of email, we need SMS:

```
public class SmsService {

    public void sendSms() {
        System.out.println("SMS notification sent to customer");
    }
}
```

We must modify `OrderService`:

```
public class OrderService {

    private SmsService smsService = new SmsService();

    public void placeOrder() {
        System.out.println("Order placed successfully");
        smsService.sendSms();
    }
}
```

What is the real problem?

The problem is **not** that the notification requirement changed.

The problem is:

> **A change in notification logic forced us to modify `OrderService`.**

Therefore:

```
Notification implementation changes
              |
              v
OrderService must change
              |
              v
Tight coupling
```

---

5. What Exactly Is Tight Coupling?

Definition

> **Tight coupling means one class is directly dependent on a specific concrete class.**

Here:

```
OrderService
      |
      v
EmailService
```

`OrderService` directly knows and creates `EmailService`.

This makes the code harder to:

- Change
- Test
- Reuse

---

6. First Improvement — Use an Interface

Instead of depending directly on `EmailService`, introduce an abstraction:

```
public interface NotificationService {

    void sendNotification();
}
```

Now multiple implementations can implement it.

Email implementation

```
public class EmailService implements NotificationService {

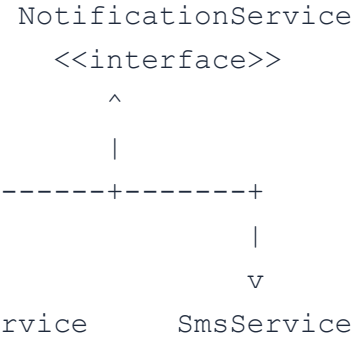
    @Override
    public void sendNotification() {
        System.out.println("Email sent");
    }
}
```

SMS implementation

```
public class SmsService implements NotificationService {

    @Override
    public void sendNotification() {
        System.out.println("SMS sent");
    }
}
```

Now we have:



`OrderService` can now depend on the interface:

```
public class OrderService {

    private NotificationService notificationService =
            new EmailService();

    public void placeOrder() {
        System.out.println("Order placed successfully");
        notificationService.sendNotification();
    }
}
```

This is better because the **variable type** is now:

```
NotificationService
```

instead of:

```
EmailService
```

But there is still a problem.

---

7. Interface Alone Is Not Enough

Look carefully:

```
private NotificationService notificationService =
        new EmailService();
```

Although we use the interface type, we still have:

```
new EmailService()
```

Why is this still a problem?

Because `OrderService` is still deciding:

> "I want to create and use EmailService."

So:

```
OrderService
     |
     | depends on interface
     v
NotificationService
     ^
     |
     | BUT creates
     |
new EmailService()
```

The object creation is still controlled by `OrderService`.

Important principle

> **Creating objects is not the problem. Creating them in the wrong place is the problem.**

`OrderService` should focus on **order-related business logic**.

It should not be responsible for deciding which notification implementation to create.

Currently `OrderService` is doing two jobs:

1. Handling order-related logic.
2. Creating/deciding the notification service.

The PDF connects this problem with two design principles:

- **S — Single Responsibility Principle**
- **O — Open/Closed Principle**

Core idea

> **Dependency is normal. Tight dependency is the problem.**

---

8. Solution — Dependency Injection

Instead of creating the dependency inside `OrderService`, provide it from outside.

```
public class OrderService {

    private NotificationService notificationService;

    public OrderService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    public void placeOrder() {
        System.out.println("Order placed successfully");
        notificationService.sendNotification();
    }
}
```

Now `OrderService` does **not** create:

```
EmailService
SmsService
```

It simply says:

> **"Give me something that can send a notification."**

That "something" is represented by:

```
NotificationService
```

---

9. Object Creation Happens Outside

Now object creation happens outside `OrderService`.

For example, `Main` can create and wire the objects:

```
public class Main {

    public static void main(String[] args) {

        NotificationService notificationService =
                new EmailService();

        OrderService orderService =
                new OrderService(notificationService);

        orderService.placeOrder();
    }
}
```

Flow:

```
             Main
              |
              | creates
              v
        EmailService
              |
              | provided to
              v
        OrderService
              |
              v
       placeOrder()
```

Now:

```
OrderService
     |
     | depends on
     v
NotificationService
```

but:

```
Main
 |
 | decides implementation
 +------> EmailService
 |
 +------> injects it into OrderService
```

Therefore:

- `OrderService` does not create `EmailService`.
- `OrderService` receives its dependency from outside.
- `OrderService` depends on the abstraction.

This is **Dependency Injection**.

---

10. What Is Dependency Injection?

Definition

> **Dependency Injection means a class receives the objects it depends on from outside, instead of creating those objects itself.**

Easy way to remember

> **Do not create your dependency. Receive your dependency.**

Or:

> **A class should ask for what it needs, not build everything by itself.**

Important

Dependency Injection is **not only a Spring concept**.

It is a **design principle** that can be used in plain Java.

Spring did not invent Dependency Injection.

> **Spring automates Dependency Injection.**

---

11. Benefits of Dependency Injection

Benefit 1 — Easy to Change Implementation

Suppose we want SMS instead of email.

We don't change `OrderService`.

We only change what is passed from outside:

```
NotificationService notificationService =
        new SmsService();

OrderService orderService =
        new OrderService(notificationService);
```

`OrderService` remains unchanged.

Flow:

```
Same OrderService
       |
       +--> EmailService
       |
       +--> SmsService
       |
       +--> Another implementation
```

Benefit 2 — Easier to Test

For testing, we may not want to send a real email or SMS.

We can create a fake implementation:

```
public class FakeNotificationService
        implements NotificationService {

    @Override
    public void sendNotification() {
        System.out.println("Fake notification for testing");
    }
}
```

Then:

```
NotificationService fake =
        new FakeNotificationService();

OrderService orderService =
        new OrderService(fake);
```

Now we can test `OrderService` without using real email/SMS logic.

Flow:

```
OrderService
     |
     v
NotificationService
     ^
     |
FakeNotificationService
     |
     v
Safe testing
```

Benefit 3 — More Reusable Code

The same `OrderService` can work with:

```
EmailService
SmsService
WhatsAppService
FakeNotificationService
```

This makes the design:

- Flexible
- Reusable
- Easier to change
- Easier to test

---

12. Types of Dependency Injection

The PDF covers three common types:

1. Constructor Injection
2. Setter Injection
3. Field Injection

---

13. Constructor Injection

The dependency is provided through the constructor.

```
public class OrderService {

    private NotificationService notificationService;

    public OrderService(
            NotificationService notificationService) {

        this.notificationService = notificationService;
    }
}
```

Flow:

```
Outside
   |
   | dependency
   v
Constructor
   |
   v
OrderService
```

Why is Constructor Injection preferred?

The PDF notes that constructor injection is usually preferred because it makes **required dependencies clear**.

If `OrderService` cannot work without `NotificationService`, constructor injection is a good choice.

Recall

> **Required dependency → Constructor Injection**

---

14. Setter Injection

The dependency is provided through a setter method.

```
public class OrderService {

    private NotificationService notificationService;

    public void setNotificationService(
            NotificationService notificationService) {

        this.notificationService = notificationService;
    }

    public void placeOrder() {
        System.out.println("Order placed successfully");
        notificationService.sendNotification();
    }
}
```

Flow:

```
OrderService created
        |
        v
Setter called
        |
        v
Dependency provided
```

Setter injection is useful when:

- The dependency is optional.
- The dependency can be changed later.

Recall

> **Optional/changeable dependency → Setter Injection**

---

15. Field Injection

The PDF introduces **Field Injection** as another type and notes that it will be discussed later in Spring.

The key recall point from this PDF is:

```
Dependency Injection types

        DI
        |
    +---+---+----------------+
    |       |                |
Constructor Setter          Field
```

---

16. What Is IoC?

IoC = Inversion of Control

To understand IoC, ask:

> **Who controls the creation of `EmailService`?**

Earlier code

```
class OrderService {

    private EmailService emailService =
            new EmailService();
}
```

Here:

```
OrderService
     |
     | controls creation
     v
EmailService
```

So the control is **inside** `OrderService`.

---

17. After Dependency Injection

After DI:

```
public class OrderService {

    private NotificationService notificationService;

    public OrderService(
            NotificationService notificationService) {

        this.notificationService = notificationService;
    }
}
```

Now `OrderService` does not create the dependency.

The object is created outside and passed into it.

```
Outside
   |
   | creates dependency
   v
EmailService
   |
   | injects
   v
OrderService
```

The control of object creation moved from inside the class to outside the class.

That movement is called:

> **Inversion of Control (IoC)**

---

18. Simple Definition of IoC

Earlier

> The class created what it needed.

```
OrderService
     |
     | creates
     v
EmailService
```

Now

> The class receives what it needs.

```
Main
 |
 | creates
 v
EmailService
 |
 | provides
 v
OrderService
```

The control has been reversed.

That's why it is called **Inversion of Control**.

---

19. IoC Flow — Before vs After

Before DI

```
OrderService
      |
      | creates
      v
EmailService
```

`OrderService` controls creation.

After DI

```
Main
 |
 | creates
 v
EmailService
 |
 | provides
 v
OrderService
```

Control has moved outside `OrderService`.

---

20. Relationship Between IoC and DI

IoC and DI are closely related, but they are **not exactly the same**.

IoC

> **IoC → Principle**

It is the idea that control is moved from the class itself to an external mechanism.

DI

> **DI → Technique**

It is one technique used to achieve IoC.

```
IoC
 |
 | principle
 v
Control moves outside the class
 |
 | one way to achieve it
 v
Dependency Injection
```

Easy interview answer

> **IoC is the principle; Dependency Injection is a technique used to implement that principle.**

---

21. Where Does Spring Fit In?

In plain Java, we moved object creation outside `OrderService`.

But now `Main` is responsible for:

```
NotificationService notificationService =
        new EmailService();

OrderService orderService =
        new OrderService(notificationService);
```

This is fine for a small application.

But imagine an application with hundreds of classes and dependencies.

```
Class A -> B -> C
Class D -> E -> F
Class G -> H -> I
...
Hundreds of objects
```

If `Main` manually creates and connects everything, it becomes messy.

We need something that can:

1. Create objects
2. Manage objects
3. Connect objects together

This is where Spring comes in.

---

22. Spring IoC Container

Spring provides an external system called the:

> **Spring IoC Container**

The Spring IoC Container is responsible for:

- Creating objects
- Managing objects
- Injecting dependencies
- Connecting classes together

Plain Java

```
Main
 |
 +--> creates EmailService
 |
 +--> creates OrderService
 |
 +--> injects EmailService
```

Spring

```
Spring IoC Container
        |
        +--> creates EmailService
        |
        +--> creates OrderService
        |
        +--> injects EmailService
        |
        +--> manages objects
```

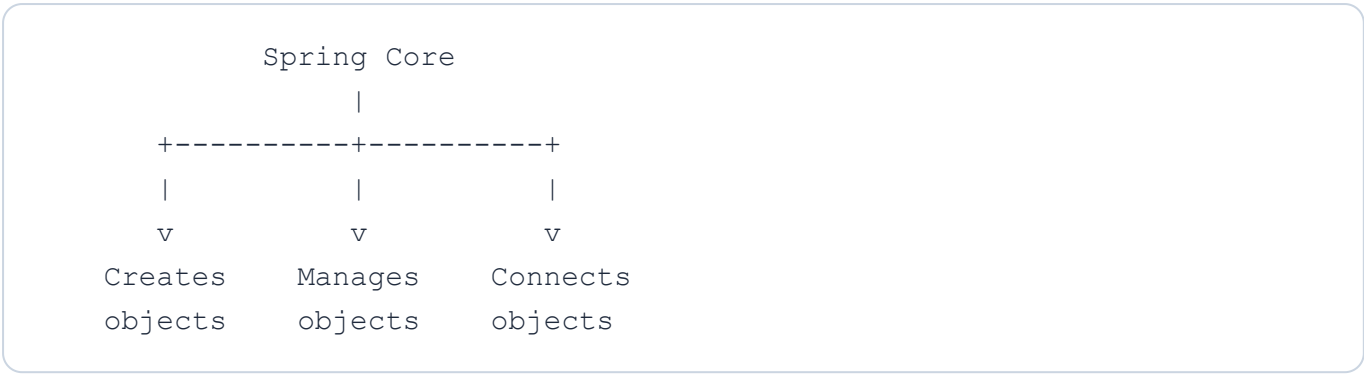
Important mental model

> **In plain Java, `Main` acted like a small container. In Spring, the Spring IoC Container performs this work automatically.**

---

23. What Does Spring Core Do?

At a high level, Spring Core does three major things:



1. Creates objects

Spring creates objects managed by its container.

2. Manages objects

Spring manages those objects and their relationships.

3. Connects objects

Spring injects dependencies and connects classes together.

This is the foundation of Spring.

Before learning:

- Spring MVC
- Spring Boot
- Spring Data JPA
- Spring Security

it is important to understand these Spring Core concepts.

---

24. What Is a Bean?

In normal Java, we usually call them:

> **Objects**

In Spring, objects managed by Spring are called:

> **Beans**

Important rule

> **Every Spring bean is an object, but every object is not necessarily a Spring bean.**

Example:

```
EmailService emailService =
        new EmailService();
```

This is a normal Java object.

If Spring creates and manages the `EmailService` object, then it becomes a:

> **Spring Bean**

---

25. Object vs Spring Bean

```
Java Object
     |
     +----------------------+
     |                      |
     v                      v
Managed by Spring?       Not managed by Spring
     |                      |
    YES                    NO
     |                      |
     v                      v
Spring Bean              Normal Object
```

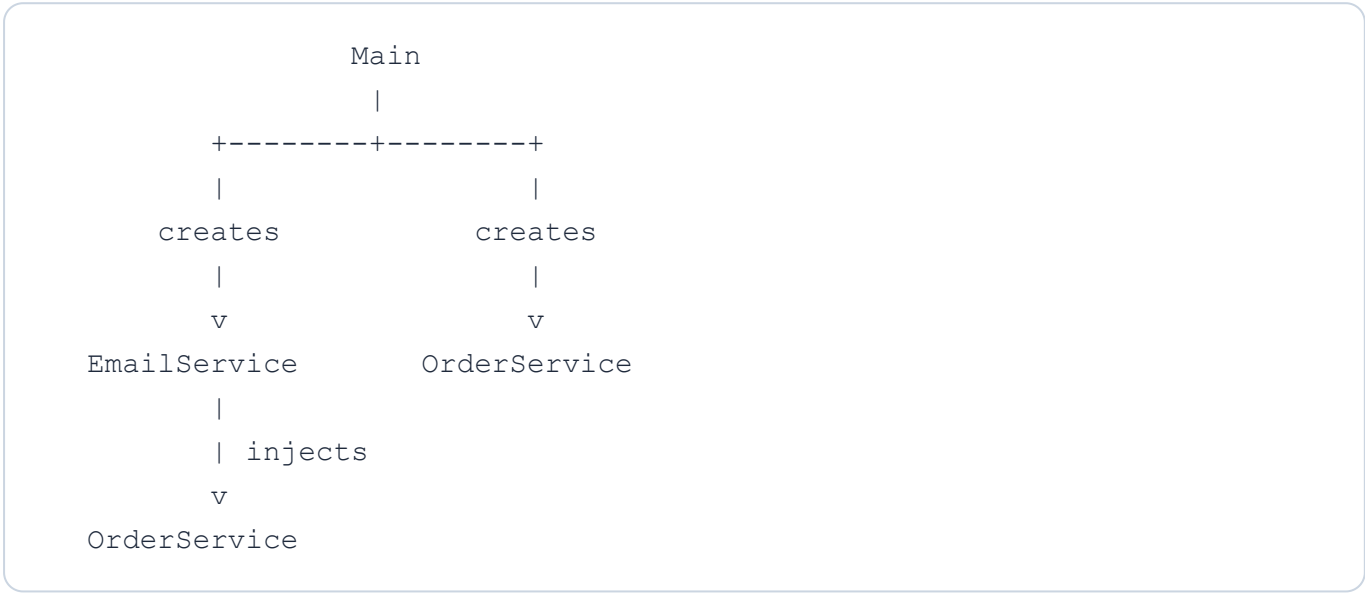
Easy recall

> **Bean = Object managed by Spring.**

26. Manual Java Flow vs Spring Flow

Plain Java Flow

In plain Java:



More simply:

```
Main
 |
 +--> creates EmailService
 |
 +--> creates OrderService
 |
 +--> injects EmailService into OrderService
```

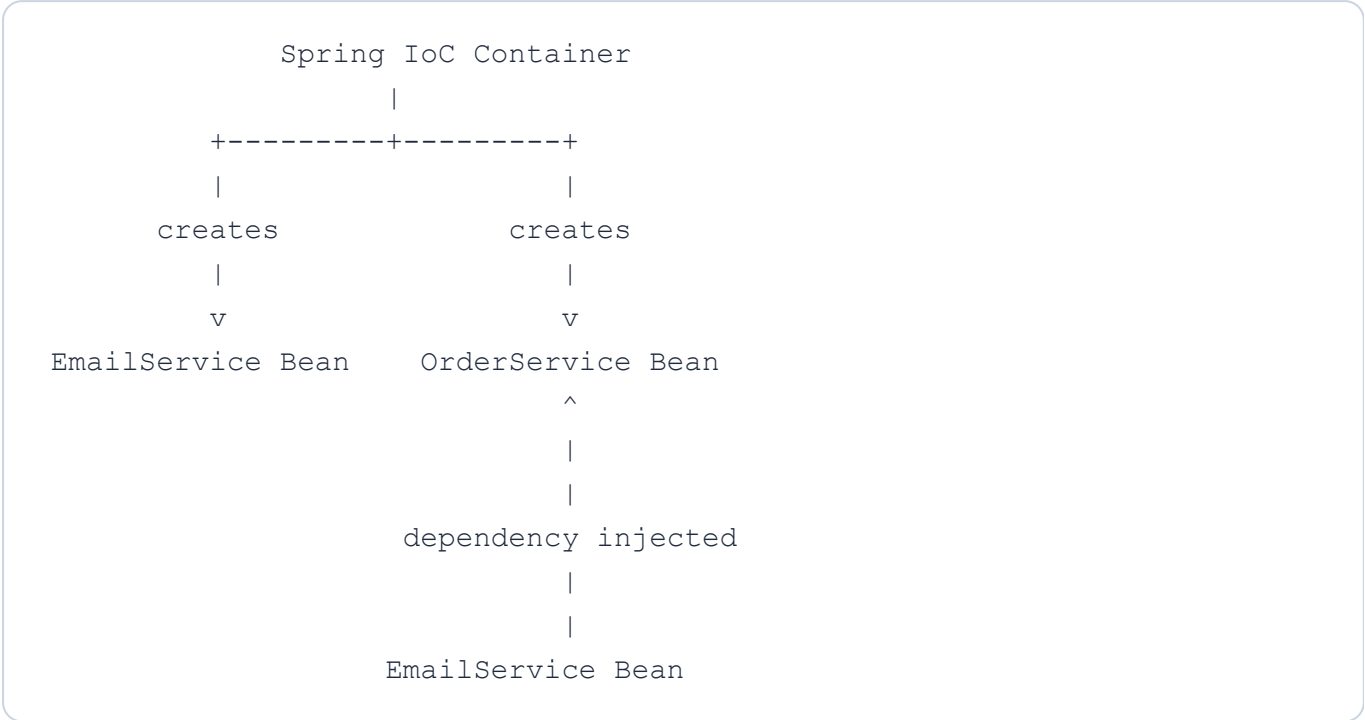
Responsibility

`Main` is responsible for:

- Creating objects
- Connecting objects
- Injecting dependencies

27. Spring Flow

In Spring:



The Spring IoC Container:

- Creates objects
- Manages objects
- Connects objects
- Injects dependencies

So the application code does not need to manually wire everything.

---

28. Complete Concept Flow

The entire topic can be remembered as one chain:

```
Direct object creation
        |
        v
Tight coupling
        |
        v
Use abstraction/interface
        |
        v
Still using new inside dependent class
        |
        v
Dependency Injection
        |
        v
Control moves outside class
        |
        v
Inversion of Control
        |
        v
Manual wiring becomes difficult at scale
        |
        v
Spring IoC Container
        |
        +--> Creates objects
        +--> Manages objects
        +--> Injects dependencies
        +--> Connects objects
        |
        v
Spring Beans
```

---

29. Important Design Evolution

This is one of the most important flows to remember for interviews.

Step 1 — Concrete dependency

```
class OrderService {

    private EmailService emailService =
            new EmailService();
}
```

Problem:

```
OrderService ---> EmailService
```

Tightly coupled.

Step 2 — Interface

```
interface NotificationService {
    void sendNotification();
}
```

Now:

```
OrderService
     |
     v
NotificationService
     ^
     |
 +---+---+
 |       |
Email   SMS
```

Better abstraction.

Step 3 — Dependency Injection

```
OrderService(NotificationService service) {
    this.notificationService = service;
}
```

Now:

```
OrderService
     |
     v
NotificationService
```

Implementation is supplied from outside.

Step 4 — IoC

Control of object creation moves outside `OrderService`.

```
Outside
   |
   +--> creates implementation
   |
   +--> injects dependency
   |
   v
OrderService
```

Step 5 — Spring

Spring automates object creation, management and wiring.

```
Spring IoC Container
        |
        +--> create
        +--> manage
        +--> inject
        +--> connect
```

---

30. Important Interview Distinctions

Dependency

> Something a class needs to complete its work.

Example:

```
OrderService needs NotificationService
```

Tight Coupling

> A class is directly dependent on a specific concrete class.

Example:

```
new EmailService()
```

inside `OrderService`.

Loose Coupling

> A class depends on an abstraction rather than a specific implementation.

Example:

```
NotificationService
```

Dependency Injection

> A class receives its dependencies from outside instead of creating them itself.

Memory line:

> **Don't create it. Receive it.**

IoC

> Control of object creation moves from the class itself to an external system.

Spring IoC Container

> Spring's external system that creates, manages, injects and connects objects.

Bean

> An object managed by Spring.

31. Interview Questions and Quick Answers

Q1. Why do we need Spring Core if Java already has classes and objects?

Spring Core helps manage the complexity of object creation, dependencies and object wiring as applications grow.

Q2. What is a dependency?

A dependency is something a class needs to complete its work.

Q3. What is tight coupling?

Tight coupling means one class is directly dependent on a specific concrete class.

Q4. Why is tight coupling a problem?

It makes the code harder to change, test and reuse.

Q5. Does using an interface automatically make code loosely coupled?

Not necessarily.

If the class still creates a concrete implementation:

```
NotificationService service =
        new EmailService();
```

then the class still controls the implementation choice.

Q6. What is the problem with `new EmailService()` inside `OrderService`?

`OrderService` is responsible for both business logic and deciding/creating its dependency. A change in notification implementation can force changes to `OrderService`.

Q7. What is Dependency Injection?

Dependency Injection means a class receives the objects it depends on from outside instead of creating them itself.

Q8. Is Dependency Injection a Spring concept?

No.

DI is a general design principle that can be used in plain Java. Spring automates it.

Q9. What are the types of Dependency Injection?

The PDF covers:

1. Constructor Injection
2. Setter Injection
3. Field Injection

Q10. Why is Constructor Injection preferred?

It makes required dependencies clear. If a class cannot work without a dependency, constructor injection is a good choice.

Q11. When is Setter Injection useful?

When the dependency is optional or can be changed later.

Q12. What is IoC?

IoC stands for Inversion of Control. It means control of object creation moves from the class itself to an external system.

Q13. What is the relationship between IoC and DI?

> **IoC is the principle; DI is one technique to achieve it.**

Q14. What does Spring IoC Container do?

It:

- Creates objects
- Manages objects
- Injects dependencies
- Connects classes together

Q15. What is a Spring Bean?

An object created and managed by Spring.

Q16. Is every Java object a Spring Bean?

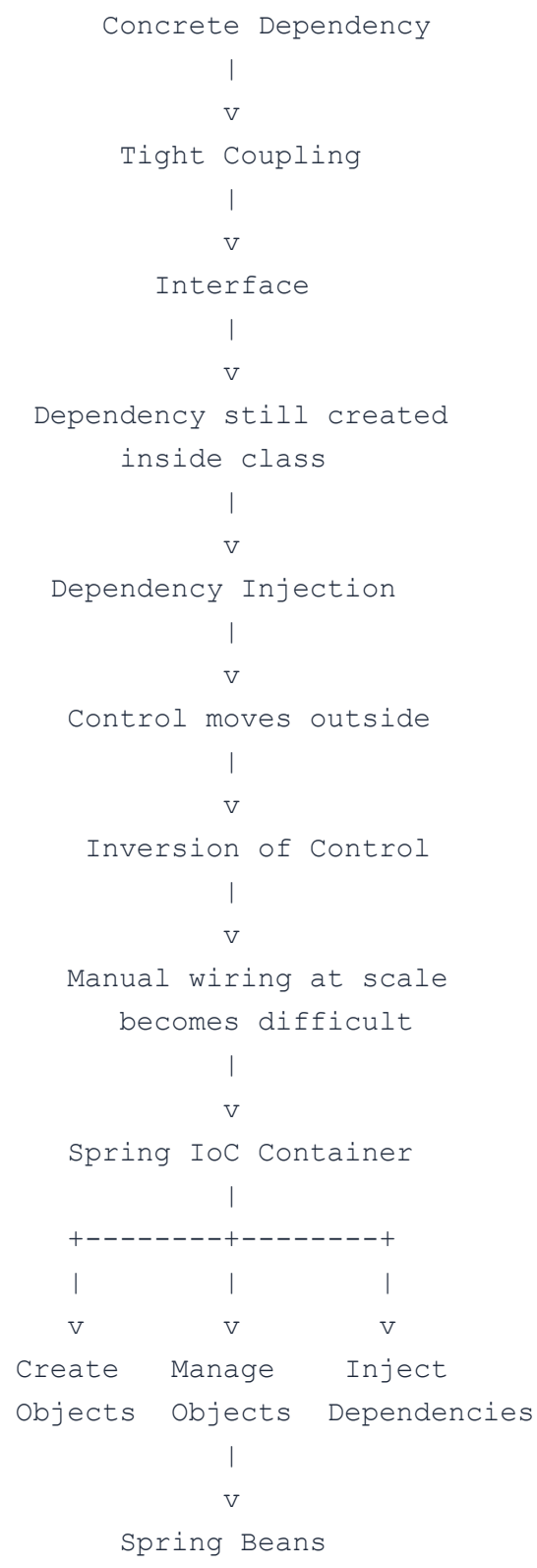
No.

> Every Spring bean is an object, but every object is not necessarily a Spring bean.

---

32. Notebook Recall Diagram

Write this diagram in your notebook because it connects almost the entire topic:


Concrete Dependency
Tight Coupling
Interface
Dependency still created
inside class
Dependency Injection
Control moves outside
Inversion of Control
Manual wiring at scale
becomes difficult
Spring IoC Container
Create
Objects
Manage
Objects
Inject
Dependencies
Spring Beans

33. Final Quick Revision

30-Second Revision

```
Dependency
    ↓
Something a class needs.

Tight Coupling
    ↓
Class directly depends on concrete implementation.

Interface
    ↓
Provides abstraction.

DI
    ↓
Dependency is provided from outside.

IoC
    ↓
Control of object creation moves outside the class.

Spring IoC Container
    ↓
Creates + manages + injects + connects objects.

Bean
    ↓
Object managed by Spring.
```

34. One-Line Definitions for Fast Recall

Concept	One-line recall
Dependency	Something a class needs to do its work
Tight Coupling	Direct dependency on a specific concrete class
Loose Coupling	Depend on abstraction rather than concrete implementation
Interface	Abstraction that allows multiple implementations
Dependency Injection	Receive dependency instead of creating it
Constructor Injection	Dependency supplied through constructor
Setter Injection	Dependency supplied through setter
Field Injection	Dependency supplied directly to a field
IoC	Control of object creation moves outside the class
Spring IoC Container	Creates, manages, injects and connects Spring objects
Bean	Object managed by Spring

---

35. Final Mental Model

Remember this sentence:

> **Java lets us create objects. Good design says a class should not create all of its own dependencies. DI lets us provide those dependencies from outside. This moves control outside the class (IoC). Spring provides the IoC Container to automate object creation, management and wiring. Objects managed by Spring are called Beans.**

```
Java
 |
 +--> Objects
 |
 v
Problem: classes create their own dependencies
 |
 v
Tight Coupling
 |
 v
Abstraction / Interface
 |
 v
Dependency Injection
 |
 v
Inversion of Control
 |
 v
Spring IoC Container
 |
 +--> Create
 +--> Manage
 +--> Inject
 +--> Connect
 |
 v
Spring Beans
```

Final Takeaway

> **Spring Core is fundamentally about managing objects and their dependencies.**
>
> **DI reduces tight coupling.**
>
> **IoC means control is moved outside the class.**
>
> **Spring IoC Container automates that control.**
>
> **A Spring-managed object is called a Bean.**